MySQL Aggregates — Quick Reference

COUNT · SUM · AVG · MIN · MAX · GROUP BY · HAVING · General reference for any MySQL database

COUNT | SUM | AVG | MIN / MAX | GROUP BY | HAVING | Query Order

What is an Aggregate Function?

Aggregate functions take a whole **group of rows** and collapse them into a **single value**. Instead of showing every row, they summarize the data.

Example: Instead of listing all 77 product prices, `AVG(UnitPrice)` gives you one number — the average.

⚠️ **Key rule:** Every non-aggregate column in SELECT must appear in GROUP BY.

Basic pattern:

```
SELECT   column,
         AGG_FUNC(other_col)
FROM     table
WHERE    row_filter        -- optional
GROUP BY column
HAVING   AGG_FUNC(...) condition
ORDER BY column;
```

The Five Aggregate Functions

Each one collapses many rows into one value per group

Function	What it does	Example usage	Handles NULLs how?
`COUNT(*)`	Counts every row in the group	`COUNT(*) → 77`	*Includes NULL rows*
`COUNT(col)`	Counts non-NULL values in that column	`COUNT(Region) → 4`	*Skips NULLs*
`COUNT(DISTINCT col)`	Counts unique non-NULL values only	`COUNT(DISTINCT CustomerID) → 89`	*Skips NULLs and dupes*
`SUM(col)`	Adds up all values in the column	`SUM(Freight) → 64942.69`	*Skips NULLs*
`AVG(col)`	Mean value — sum divided by non-NULL count	`AVG(UnitPrice) → 28.87`	*Skips NULLs*
`MIN(col)`	Smallest value in the group	`MIN(UnitPrice) → 2.50`	*Skips NULLs*
`MAX(col)`	Largest value in the group	`MAX(UnitPrice) → 263.50`	*Skips NULLs*

GROUP BY — split rows into groups

What it does: Splits rows into groups by a column, then runs the aggregate on each group separately — giving one result row per group.

```
SELECT   CategoryID,
         COUNT(*) AS 'NumProducts'
FROM     products
GROUP BY CategoryID;   -- one row per CategoryID
```

> *Without GROUP BY, COUNT(*) returns one number for the whole table. With it, you get one number per group.*

The GROUP BY Rule

Every column in **SELECT** that is **not inside** an aggregate function must be listed in **GROUP BY**. MySQL will error otherwise.

```
-- WRONG — CategoryID not in GROUP BY
SELECT CategoryID, COUNT(*)
FROM   products;

-- CORRECT
SELECT CategoryID, COUNT(*)
FROM   products
GROUP BY CategoryID;
```

> *Error message: "not in GROUP BY clause" — check every non-aggregate SELECT column.*

MySQL Aggregates — Page 2

HAVING vs WHERE · Query execution order · ROUND() · Aggregates with JOINs

HAVING vs WHERE — the most important distinction

Both filter rows — but at completely different stages of the query

WHERE

Filters **individual rows BEFORE** grouping. Runs on raw table data. Cannot use aggregate functions here.

```
SELECT CategoryID, COUNT(*)
FROM   products
WHERE  UnitPrice > 20   -- filter rows first
GROUP BY CategoryID;
-- only counts products priced over $20
```

WHERE COUNT(*) > 5 → ERROR — cannot use aggregates in WHERE

HAVING

Filters **groups AFTER** grouping. Runs on aggregated results. Must use aggregate functions here.

```
SELECT CategoryID, COUNT(*)
FROM   products
GROUP BY CategoryID
HAVING COUNT(*) > 5;  -- filter groups after
-- only categories with 5+ products
```

HAVING COUNT(*) > 5 — works correctly, filters after groups are formed

Easy rule: Filtering a **regular column**? Use `WHERE` | Filtering an **aggregate result**? Use `HAVING`

Query Execution Order — MySQL processes clauses in this order internally

NOT the order you write them. Explains why aliases work in ORDER BY but not WHERE.

1	FROM / JOIN	Load and combine tables
2	WHERE	Filter individual rows (before grouping)
3	GROUP BY	Split remaining rows into groups
4	HAVING	Filter groups (after grouping)
5	SELECT	Choose columns + run aggregates — aliases created here
6	DISTINCT	Remove duplicate rows
7	ORDER BY	Sort final results — aliases work here
8	LIMIT	Cut to N rows

Aliases are created at step 5. ORDER BY runs at step 7 — after aliases exist. WHERE runs at step 2 — before aliases exist. That is why aliases work in ORDER BY but NOT in WHERE.

ROUND() — clean up decimal results

AVG and SUM often return many decimal places. Wrap with ROUND(value, decimal_places) to clean up output.

```
-- Without ROUND:
AVG(UnitPrice) → 28.8663265...

-- With ROUND:
SELECT ROUND(AVG(UnitPrice), 2)
         AS 'AvgPrice'
FROM   products;
-- Returns: 28.87

-- ROUND(value, 0) = whole number
-- ROUND(value, 2) = 2 decimal places
```

Aggregates + JOINs together

JOIN goes between FROM and WHERE as usual. GROUP BY and HAVING then work on the joined result.

```
SELECT  c.CategoryName,
        COUNT(*) AS 'NumProducts',
        ROUND(AVG(p.UnitPrice),2)
                 AS 'AvgPrice'
FROM    products   p
JOIN    categories c
  ON    p.CategoryID = c.CategoryID
GROUP BY c.CategoryName
HAVING  COUNT(*) > 5
ORDER BY 'AvgPrice' DESC;
```

Line-by-Line Syntax Breakdowns

Real aggregate query scenarios — inline comments explain every line

```sql
SELECT  c.CategoryName,                    -- group label — must be in GROUP BY below
        COUNT(p.ProductID) AS 'Total'        -- count how many products per category
FROM    products   p                         -- starting table, alias p
JOIN    categories c                         -- join to get CategoryName instead of just ID
  ON    p.CategoryID = c.CategoryID          -- shared key connecting the two tables
GROUP BY c.CategoryName                     -- one result row per unique CategoryName
HAVING  COUNT(p.ProductID) > 5             -- filter AFTER grouping: drop groups with 5 or fewer products
ORDER BY 'Total' DESC;                      -- sort by count, largest first
```

```sql
SELECT  o.CustomerID,                           -- group label — must be in GROUP BY
        COUNT(o.OrderID)     AS 'NumOrders',     -- count orders per customer group
        ROUND(SUM(o.Freight),2) AS 'TotalFreight'  -- sum of freight, rounded to 2 decimal places
FROM    orders o                                -- source table, alias o
WHERE   o.ShipCountry = 'Germany'              -- runs at step 2: filter rows BEFORE grouping
GROUP BY o.CustomerID                           -- group the Germany orders by customer
HAVING  COUNT(o.OrderID) > 3                   -- runs at step 4: only keep groups with 3+ orders
ORDER BY 'TotalFreight' DESC;                  -- sort by freight total, largest first
```

```sql
SELECT  c.CategoryName,                         -- only non-aggregate column — goes in GROUP BY
        COUNT(*) AS 'NumProducts',               -- how many products in this category
        ROUND(AVG(p.UnitPrice), 2) AS 'AvgPrice',  -- average price, rounded 2 decimal places
        MIN(p.UnitPrice) AS 'MinPrice',          -- cheapest product in this category
        MAX(p.UnitPrice) AS 'MaxPrice',          -- most expensive product in this category
        SUM(p.UnitsInStock) AS 'TotalStock'      -- total units across all products in category
FROM    products   p                             -- starting table
JOIN    categories c                             -- join to get readable category name
  ON    p.CategoryID = c.CategoryID              -- shared key
GROUP BY c.CategoryName                          -- CategoryName is the only non-aggregate in SELECT
ORDER BY 'AvgPrice' DESC;                       -- sort by average price, most expensive first
```